

# ELEVATE 1.0



**Team Name:** Team Preflight

**Product:** Preflight. Should you believe this ML paper?

**PS ID :** EL-01

**PS Name :** AI-Powered ML Paper Reproducibility Platform

## Abstract :

**Preflight** re-runs every number in an ML paper over ten seeds, explains each gap with a counterfactual re-run, and flags results inflated by leakage, unfair baselines, wrong metrics or lucky seeds.

**Claude reads the paper; re-runs decide.** 13/13 planted issues caught, 0 false alarms.



[Open the live demo](#)

# PROPOSED SOLUTION



**The problem:** ML results can reproduce and still mislead: leaked test data, untuned baselines, lucky seeds.

**Today's tools** stop at “does it run?”. Preflight asks “should you believe it?”



| What EL-01 asks for                 | What Preflight does                                        |
|-------------------------------------|------------------------------------------------------------|
| Understand the methodology          | AI reader extracts claims and stated settings; you confirm |
| Find the implementation and config  | README, config and AST mapping down to file:line           |
| Execute in a controlled environment | Sandbox: browser WASM, subprocess or Docker; 10 seeds      |
| Compare with reported results       | Seed-aware verdict with a k-seed tolerance                 |
| Explain why results differ          | Shapley gap attribution and missing-detail inference       |
| Go beyond code exists               | Leakage, metric, fair-baseline and fragility re-runs       |
| Evidence-backed, per experiment     | Claim Ledger: table cell, code line, command, log, hash    |
| Lightweight CPU-only papers         | CPU triage: full, scaled or untestable, with the reason    |

THE POINT

Reproducing is not enough. We show whether to believe a result, and prove it with a re-run.

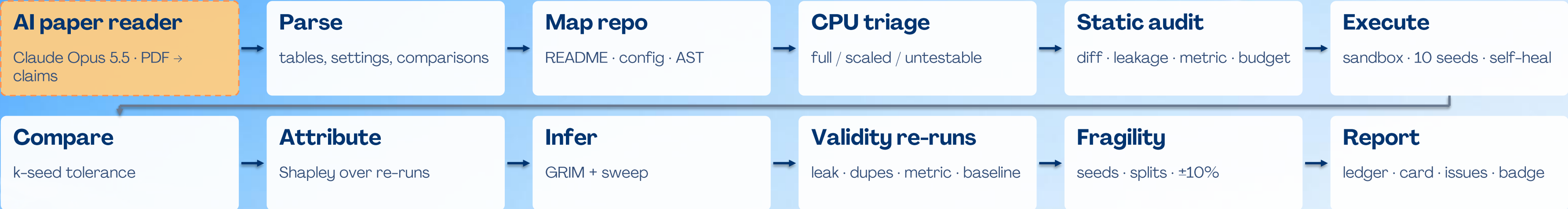
| ID | CLAIM                                                                                                                     | REPORTED → MEASURED                      | PER SEED | VERDICT        |
|----|---------------------------------------------------------------------------------------------------------------------------|------------------------------------------|----------|----------------|
| C1 | <b>Logistic Regression on Breast Cancer</b><br>Table 1 · Test accuracy (%) on Breast Cancer.<br>hyperparameter mismatch   | <b>98.25 → 95.79</b><br>± 1.23, 10 seeds |          | Not reproduced |
| C2 | <b>Decision Tree on Breast Cancer</b><br>Table 1 · Test accuracy (%) on Breast Cancer.                                    | <b>93.86 → 93.60</b><br>± 2.71, 10 seeds |          | Reproduced     |
| C3 | <b>SVM (RBF) on Breast Cancer</b><br>Table 1 · Test accuracy (%) on Breast Cancer.                                        | <b>98.25 → 97.46</b><br>± 1.13, 10 seeds |          | Reproduced     |
| C4 | <b>k-NN on Breast Cancer</b><br>Table 1 · Test accuracy (%) on Breast Cancer.<br>seed selection                           | <b>95.61 → 97.72</b><br>± 0.94, 10 seeds |          | Not reproduced |
| C5 | <b>ANOVA top-20 + Logistic Regression on SynthGene-2k</b><br>Table 2 · Test accuracy (%) on SynthGene-2k.<br>data leakage | <b>92.40 → 92.40</b><br>± 6.38, 10 seeds |          | Reproduced     |
| C6 | <b>ViT-B/16 on ImageNet-1k</b><br>Table 3 · Top-1 accuracy (%) on ImageNet-1k.<br>compute or data limit                   | <b>81.30 → -</b><br>not run              |          | Untestable     |
| C7 | <b>SVM (RBF) beats k-NN</b><br>Results text<br>seed selection Fragile Overclaim                                           | <b>2.64 → -0.26</b><br>± 1.38, 10 seeds  |          | Not reproduced |

Claim Ledger for Paper A: every number, its verdict and its cause

[Open this ledger live →](#)

# TECHNICAL APPROACH

**THE POINT**  
**One AI step.** Everything after it is deterministic, sandboxed and repeatable.



**Dashed = the only AI step (claim extraction, checked by a person).** Everything after it is deterministic: static analysis, sandboxed execution, statistics and counterfactual re-runs.

**Verdict rule**

`tol = max(2σ/√k, 0.5 pt)`  
`k = seeds paper averaged`  
`Partial: ≤ 3σ/√k`  
or a scaled run

**Gap attribution**

$$\phi_i = \sum |S|!(n-|S|-1)!/n! \cdot [v(SU\{i\}) - v(S)]$$
  
`v(S): 10-seed mean,`  
`S set to paper values`

**Sandbox**

fresh repo copy, hashed  
seed set at the code's seed site  
Docker: no network, CPU/RAM caps  
probe: fits, splits, row hashes

**Stack**

Python 3.14 · scikit-learn 1.8  
Pyodide (WASM), Web Worker  
FastAPI · queue · Docker  
Claude Opus 5.5, JSON output



The same 11 stages, live in the app

# INNOVATION AND UNIQUENESS



|                    | Runs code | Per-claim verdict | Explains why (re-run) | Leakage check | Fragility / seeds | False-alarm control | Metric & baseline audit |
|--------------------|-----------|-------------------|-----------------------|---------------|-------------------|---------------------|-------------------------|
| VERITAS (2026)     | Yes       | Yes               | Partly                | No            | No                | No                  | No                      |
| CORE-Bench agents  | Yes       | Partly            | No                    | No            | No                | No                  | No                      |
| REPRO-Bench agents | Yes       | Partly            | Partly                | No            | No                | No                  | No                      |
| Code Ocean         | Yes       | No                | No                    | No            | No                | No                  | No                      |
| SciCoQA / Dude     | No        | Partly            | Partly                | No            | No                | LLM                 | No                      |
| Preflight          | Yes       | Yes               | Yes                   | Yes           | Yes               | Re-run              | Yes                     |

Based on each tool's published description. Partly: covers part of the column.

## “How is this different from VERITAS?”

VERITAS tells you whether a claim replicates. Preflight tells you why it failed, whether a passing result is inflated, and how fragile it is, and backs each finding with a counterfactual re-run.

**Original:** a GRIM check rules out split ratios that cannot produce the reported accuracy before anything runs; seeds are injected at the code's own seed site; every flag must survive a re-run (Dude's false-positive problem).

THE POINT

Others say whether. Preflight shows why, and catches results that reproduce but mislead.

96%

of a 2.46-pt gap explained by one config value (C), via Shapley re-runs

-28.8 pt

a result that reproduces, but only because feature selection saw the test set

+1.52 → -0.56

a claimed win that flips once the baseline gets the same tuning budget

15 / 30

conditions where “significantly outperforms” still holds

13 / 13

planted issues caught across the test papers, with 0 false alarms on the clean control. One real 1993 paper audited end to end.

# FEASIBILITY AND VIABILITY



THE POINT

Built and running today. Every demo audit finishes within a minute, on a CPU, in the browser.

Every demo audit runs on a CPU in seconds [Run Paper A live](#) →

| Paper                        | Sandboxed runs | From cache | CPU-s (laptop) | Live in browser |
|------------------------------|----------------|------------|----------------|-----------------|
| A • planted (7 claims)       | 242            | 94         | 3.7            | 6 s             |
| B • clean control            | 22             | 2          | 1.5            | 2.6 s           |
| C • planted (6 claims)       | 151            | 66         | 24.4           | 55 s            |
| R • real, Street et al. 1993 | 82             | 49         | 8.6            | 13.3 s          |

Browser: Pyodide 314.0.7 in Chrome. Live and recorded runs match exactly.

## Risks and mitigations

| Risk                         | Mitigation                                                   |
|------------------------------|--------------------------------------------------------------|
| PDF tables misread           | Claude with a JSON schema, then a person confirms the claims |
| LLM hallucination            | LLM only extracts; every finding is a re-run                 |
| Repo breaks on new libraries | self-heal rules, patch shown, logged as impediment           |
| Experiment too big for CPU   | triage: scale rows to the RAM budget, verdict ≤ Partial      |
| Hostile code                 | Docker: no network, CPU/RAM/PID caps, read-only root         |
| Venue wifi fails             | recorded replay, labelled as recorded                        |

## Three ways to deploy, one engine

- Browser

Pyodide in a Web Worker. Zero server cost, nothing leaves the laptop. Works today.
- Server

FastAPI, job queue, one Docker container per run, result cache by content hash. Tested.
- CI / author mode

The CLI writes a GitHub Action that re-audits on every push, plus a README badge.

Sustainability

Open-source core (MIT) so labs can self-host and trust the checks. Hosted tier for conferences and journals: batch audits of artifact submissions, badge issuing, reviewer reports.

Compute cost scales with papers audited, and small papers cost nothing because they run in the visitor's browser.

# IMPACT AND SCALING

DEP

## Who it is for

### Reviewers & repro chairs

triage artifact submissions in minutes, with evidence to cite

### Students & MLRC

reproduce a paper for coursework and learn why it differs

### ML teams

check a paper's claims before building on them

### Authors

pre-submission check and a README badge

## Tested, not promised

- ✓ 13/13 planted issues caught, 0 false alarms (clean control: 0 findings)
- ✓ 11 automated tests: planted, clean, live edits, real paper, server API
- ✓ Browser and laptop runs give identical numbers
- ✓ Live edits re-audited: fixing C turns C1 green; a new scaler leak is caught
- ✓ Server mode: HTTP submit, poll, idempotent cache, rate limits
- ✓ Real 1993 paper: one claim exact, one flagged with the paper's own reason

### THE POINT

**Success is cheap.** Audits run in the user's browser, so 10× users is mostly static hosting.



ARR

## YC says yes tomorrow. Can it handle success next week?

### Users

Most audits run in the visitor's browser, so 10× traffic is mostly static file hosting.

### Load

Stateless API + queue; autoscaled Docker workers; one container per run.

### Repeat work

Runs cached by hash of files, command and library versions; duplicate submissions are free.

### Breaks

Per-run CPU, RAM, PID and time caps; network off; rate limit per client.

### Crashes

Self-heal known API breaks; failed jobs keep their events; static audit still reports.

### Robustness

Idempotent jobs and retries; recorded replay if live compute is down.

# RESEARCH AND REFERENCES

## THE POINT

**Still unsolved.** Top agents replicate under a third of papers; top LLMs find under half the discrepancies.



DEP

ARR

**21-27%**

best agents on PaperBench vs 41% for ML PhDs: replication is still hard (OpenAI, 2025)

**46.7%**

of real paper–code discrepancies found by the best LLMs (SciCoQA, ACL 2026)

**294+**

papers in 17 fields affected by leakage (Kapoor & Narayanan, Patterns 2023)

**68%**

full replication by VERITAS after automated fixes; it reports whether, not why (arXiv 2607.02931)

## Try it

### Live demo

[preflight-el01.github.io/preflight](https://preflight-el01.github.io/preflight)

### Code (MIT)

[github.com/preflight-el01/preflight](https://github.com/preflight-el01/preflight)

### Video

[Watch the 90-second walkthrough](#)

### Test kit

[A test paper with planted bugs](#)



In the app: press **Tour** for a guided walkthrough, Ctrl K to search, Audit your own to upload a PDF.

## Sources

Kapoor & Narayanan. Leakage and the reproducibility crisis in ML-based science. Patterns, 2023

Starace et al. PaperBench. OpenAI, 2025 · Siegel et al. CORE-Bench, 2024 · REPRO-Bench, ACL 2025

SciCoQA: paper–code alignment, ACL 2026 (arXiv 2601.12910) · Dude, arXiv 2609.03416

VERITAS: a general-purpose replication tool, arXiv 2607.02931

Brown & Heathers. The GRIM test. Social Psychological and Personality Science, 2017

Pineau et al. ML Reproducibility Checklist, 2021 · Kapoor et al. REFORMS, 2023

Street, Wolberg & Mangasarian. Nuclear feature extraction for breast tumor diagnosis. SPIE, 1993

## Datasets

Bundled with scikit-learn: Wisconsin Diagnostic Breast Cancer (569 cases), Digits. Generated: SynthGene-2k, SynthMed, SynthHD, SynthKNN-Aug, SynthLarge-2M.

## Papers

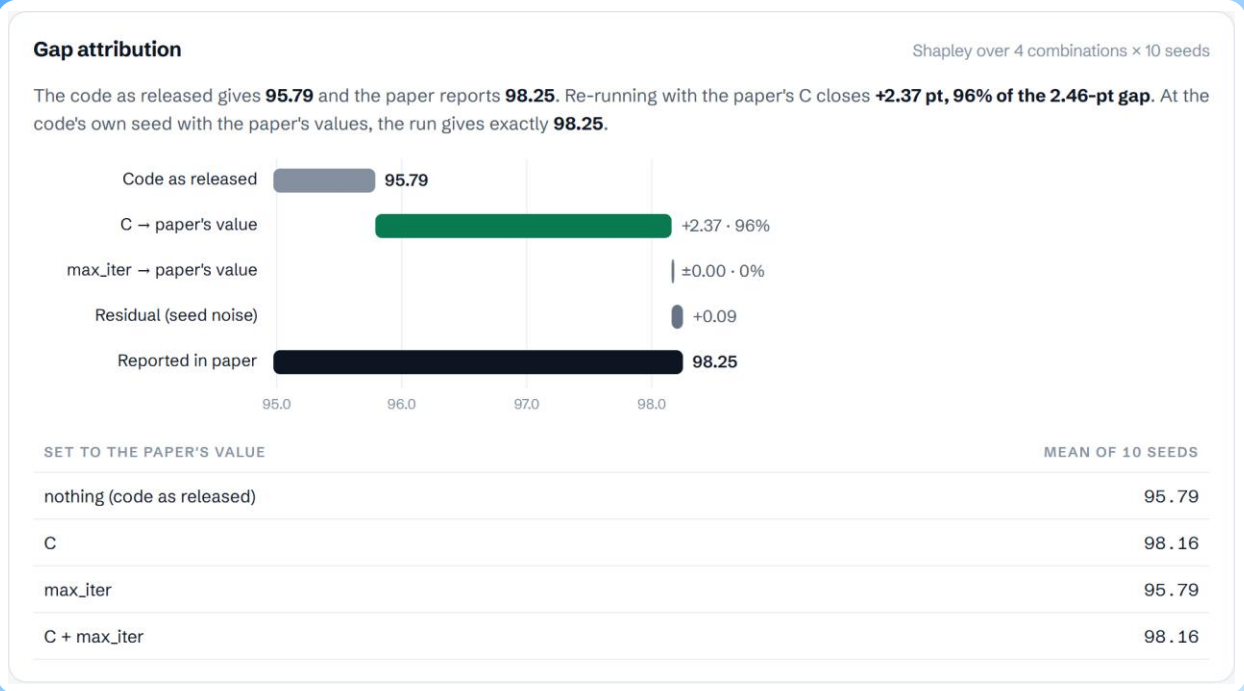
Three fictional test papers with an answer key (A, B, C) and one real paper (Street et al., 1993). Every number shown comes from a logged sandbox run.

# LIVE PROOF

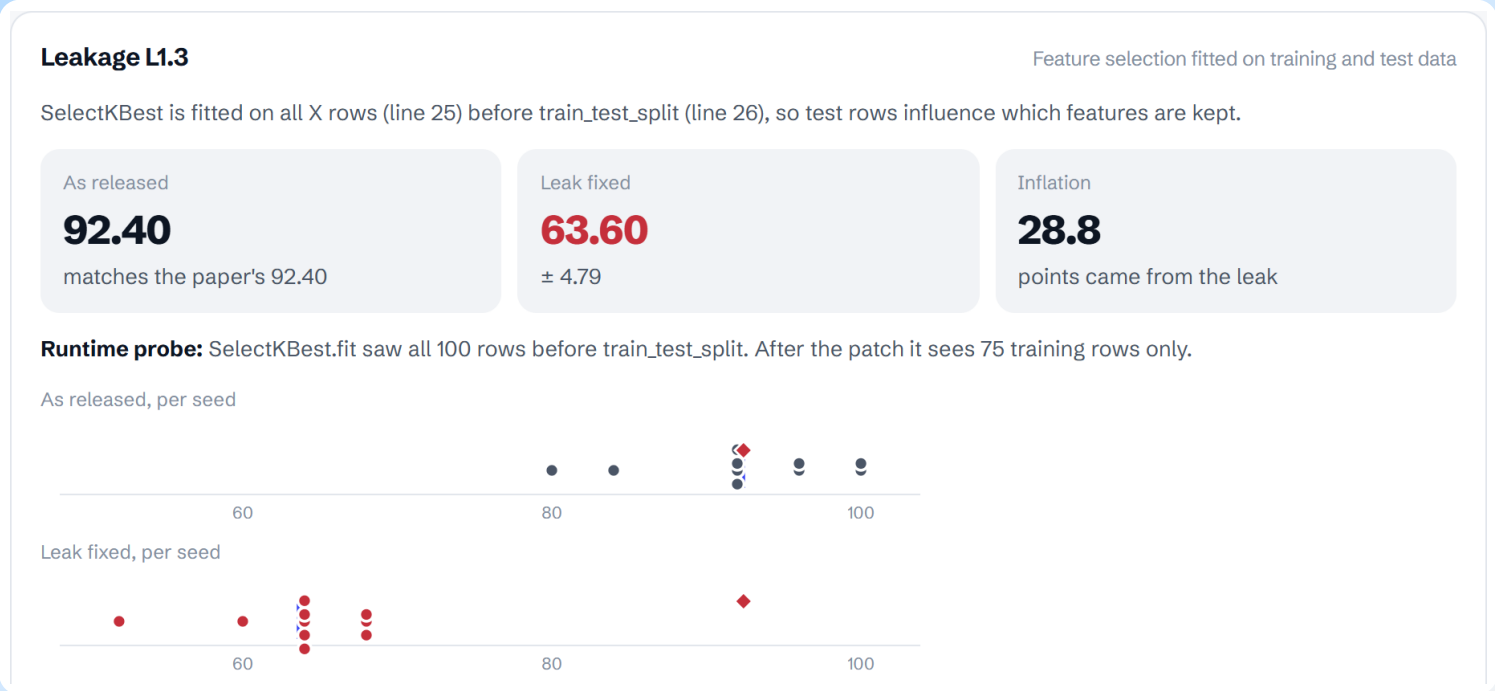


**THE POINT**  
**Proof, not promises.** Four findings from the real app, each backed by a re-run.

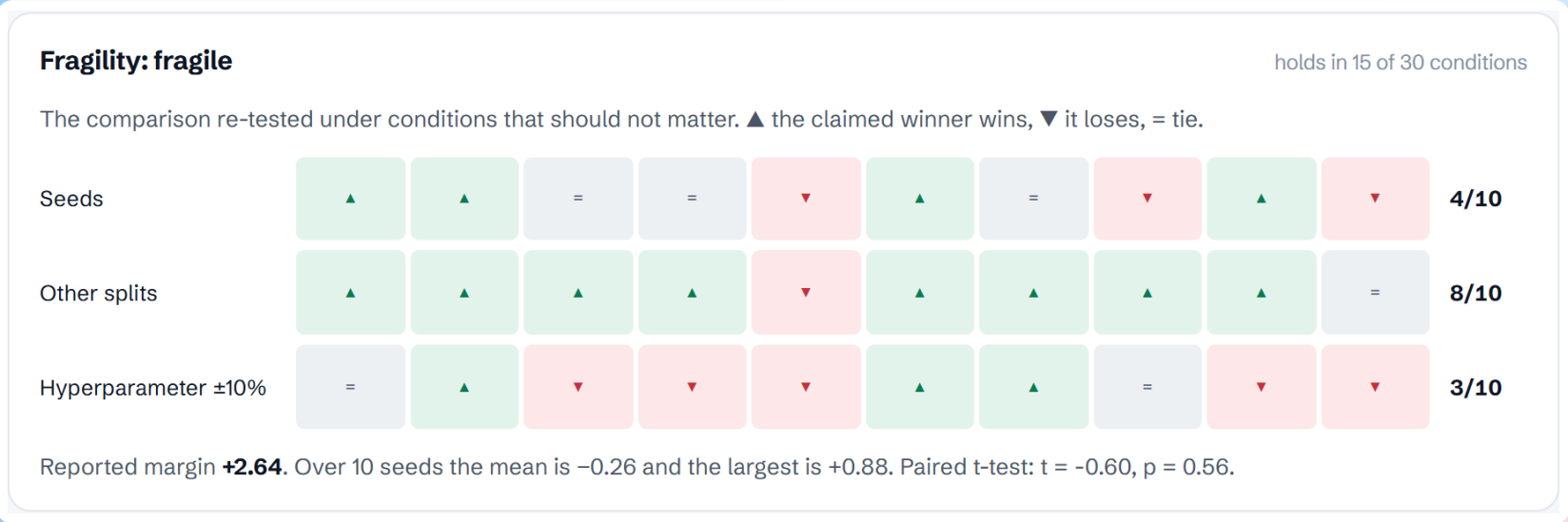
Each screen is a re-run, not an opinion. Click any one to open it live in the app.



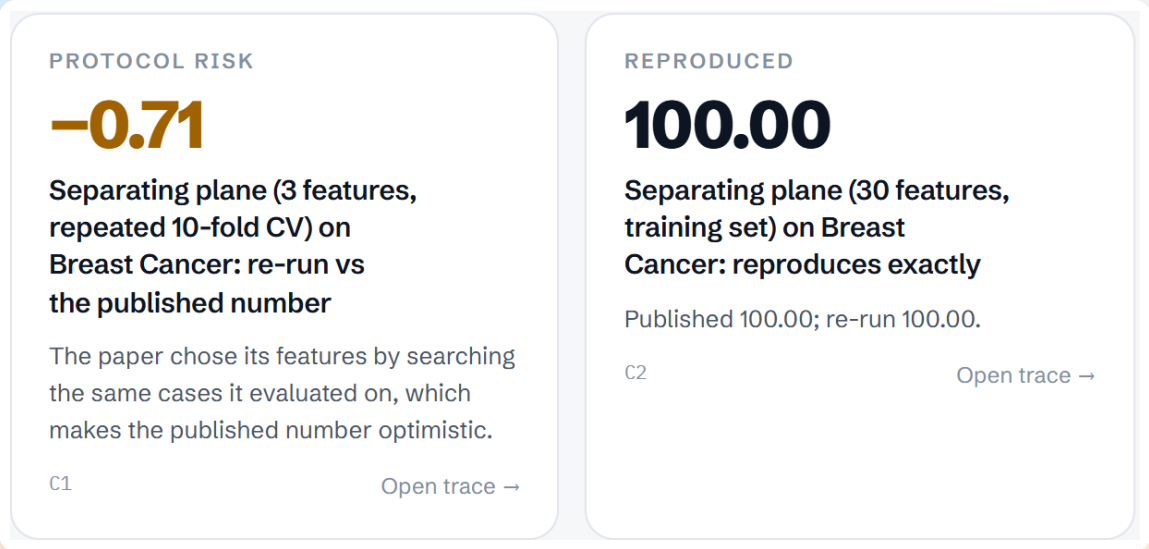
Why C1 fails: one config value explains 96% of the gap →



Why C5 cannot be trusted: the leak, the patch, the drop →



Why C7 is fragile: 15 of 30 conditions →



**Real paper**  
Street, Wolberg & Mangasarian, 1993.  
The authors released no code, so the repo is our reproduction.

A real paper: one claim exact, one optimistic, and its reason →